Smart Video Indexer

Project Report

Team Name- The Hunter

Manas Mondal & Arkaprava Jas

1. Abstract

The **Unified Video-Text Grounding (UnVTG)** system is a two-stage deep learning pipeline designed to temporally localize natural language queries within untrimmed videos. Given a video and a free-form text query, the system identifies the start and end timestamps of the video segment that best corresponds to the described event.

Stage 1 fine-tunes OpenAI's CLIP (ViT-B/32) via a contrastive objective over the ActivityNet Captions dataset, aligning visual and textual representations in a shared embedding space. A sliding-window CLIP filter then generates coarse candidate segments. **Stage 2** introduces a Query-Dependent Detection Transformer (QD-DETR) that refines those coarse proposals into precise temporal boundaries.

The unified system is evaluated using standard Video Temporal Grounding metrics: *Recall@1* at IoU thresholds of 0.3, 0.5, and 0.7, and mean IoU (mIoU).

2. Introduction

Video Temporal Grounding (VTG) — also called Natural Language Video Localization — is the task of finding the temporal segment in a video that semantically corresponds to a given text query. It bridges computer vision and natural language processing, with applications in video search, surveillance analytics, and assistive technologies.

Existing approaches often rely on pre-computed optical flow features or large proprietary models. This project explores a lightweight yet effective approach using only RGB frames and the CLIP image-text encoder as a shared backbone, augmented by a DETR-style detection head for precise temporal boundary regression.

Key Contributions

- A **contrastive fine-tuning strategy** for CLIP on video-text pairs from ActivityNet Captions.
- A **sliding-window CLIP filtering** stage that generates candidate temporal segments with a tunable similarity threshold.
- A **Query-Dependent Detection Transformer (QD-DETR)** that takes CLIP video and text features as input and predicts temporal boundaries in normalized (center, width) format.
- **Comprehensive evaluation** on both ActivityNet Captions validation splits (val_1

and val_2) with multiple metrics and visual diagnostics.

3. System Architecture

The UnVTG pipeline is organized into two sequential stages. Each stage has a clearly defined input, processing module, and output that feeds into the next.

3.1 Stage 1 — CLIP-Based Coarse Retrieval

The first stage uses the CLIP ViT-B/32 model to perform a coarse-grained temporal search over the full video. A sliding window of fixed duration scans the video, encoding each window as a mean-pooled frame embedding. The cosine similarity between each window embedding and the text query embedding is computed; windows exceeding a tunable threshold are retained. The **union** of all retained windows forms the candidate segment passed to Stage 2.

3.2 Stage 2 — QD-DETR Precise Localization

The second stage applies a Query-Dependent Detection Transformer within the candidate segment from Stage 1. A fixed number of frames (32 by default) are uniformly sampled, encoded by CLIP, and fed together with the text embedding into a Transformer encoder-decoder. The decoder produces a set of predicted (center, width) boxes in normalized coordinates $[0, 1]$. The box with the highest foreground class score is selected and mapped back to absolute timestamps.

4. Dataset

The project uses the **ActivityNet Captions** dataset, a large-scale benchmark for dense video captioning and temporal grounding. Each video is annotated with multiple natural language sentences, each paired with a start and end timestamp.

Table 1: ActivityNet Captions dataset splits used in this project.

Split	Approx. Pairs	Annotation File	Purpose
Train	37,000+	`activitynet_captions_train.json`	CLIP fine-tuning & QD-DETR training
Val-1	17,000+	`activitynet_captions_val_1.json`	Threshold tuning
Val-2	16,900+	`activitynet_captions_val_2.json`	Final held-out evaluation

Videos are stored locally in `.mp4` / `.mkv` format. The `Decord` library is used for fast GPU-accelerated frame decoding. Each annotation entry contains: `video_id`, `video` filename, `duration` (seconds), a list of `timestamps (start, end)`, and a list of corresponding `sentences`.

5. Stage 0 — Preprocessing & Setup

5.1 Environment & Dependencies

Table 2: Python libraries used in the UnVTG system.

Library	Role
`torch / torchvision`	Deep learning framework, GPU execution
`transformers`	CLIP model & processor (HuggingFace)
`decord`	Fast video frame decoding (CPU context)
`opencv-python (cv2)`	Frame resizing to 224×224
`numpy`	Array manipulation, frame indexing
`tqdm`	Progress bars during training
`scipy`	Hungarian matching (`linear_sum_assignment`)

CUDA availability is verified at startup; the device is selected automatically (`cuda` if available, else `cpu`).

5.2 Data Loading & Sample Construction

The JSON annotation file is parsed into a flat list of training samples. Each sample is a dictionary:

- `video` — absolute path to the video file
- `text` — natural language query sentence
- `start` / `end` — ground-truth timestamps in seconds
- `duration` — total video duration in seconds

Video path resolution checks for both `.mp4` and `.mkv` extensions. Samples with missing video files are skipped silently.

5.3 Frame Sampling Strategy

Frames are extracted at a fixed rate of **2 fps** within the specified temporal window using Decord's indexed batch reader. Each frame is resized to 224×224 via OpenCV bilinear interpolation. Short clips (fewer than 8 frames) are padded by tiling existing frames with the `repeat_frames` utility.

5.4 Dataset Class & DataLoader

The `ActivityNetDataset` PyTorch `Dataset` wraps the sample list. A custom `collate_fn_variable` zero-pads batches to the maximum frame count within each batch, ensuring compatibility with variable-length clips.

```
dataset    = ActivityNetDataset(samples)
dataloader = DataLoader(
    dataset,
    batch_size  = 2,
    shuffle     = True,
    num_workers = 2,
    collate_fn  = collate_fn_variable   # handles variable T per clip
)
# Output: frames -> [B, max_T, C, H, W]
#         texts  -> list of B strings
```

Listing 1: DataLoader configuration.

6. Stage 1 — Contrastive CLIP Fine-Tuning

6.1 CLIP Model Configuration

The base model is `openai/clip-vit-base-patch32`. All parameters are frozen **except** the `text_projection` and `visual_projection` layers, which are fine-tuned end-to-end. This selective unfreezing preserves rich general-purpose CLIP representations while adapting the projection heads to the video-text grounding task.

6.2 Encoding Pipeline

6.2.1 Text Encoder

Input strings are tokenized by `CLIPProcessor` (padding + truncation). The CLIP text model produces a pooled `[CLS]` embedding, which is then projected via `text_projection` to a 512-dimensional feature vector.

6.2.2 Frame Encoder

Batched frame tensors of shape $[B \times T,\, C,\, H,\, W]$ are passed through the CLIP vision model. The pooled output of each frame is projected via `visual_projection`, producing per-frame features of dimension 512, reshaped to $[B,\, T,\, 512]$.

6.2.3 Top-K Frame Selection

Before pooling, a **top-k selection** ($k = 8$) retains only the frames with the highest cosine similarity to the text query, discarding distractors:

$$\text{sim}_{b,t} = \langle \mathbf{f}_t^{(b)},\, \mathbf{q}^{(b)} \rangle, \qquad \mathcal{S}^{(b)} = \underset{t}{\text{top-}k}\ \text{sim}_{b,t} \tag{1}$$

6.2.4 Attention Pooling

Selected frame features are pooled using soft attention driven by text-query similarity:

$$\alpha_{b,t} = \frac{\exp(\text{sim}_{b,t})}{\sum_{t'} \exp(\text{sim}_{b,t'})}, \qquad \mathbf{v}^{(b)} = \sum_{t \in \mathcal{S}^{(b)}} \alpha_{b,t}\, \mathbf{f}_t^{(b)} \tag{2}$$

This allows the model to emphasize semantically relevant frames during the forward pass.

6.3 Contrastive Loss (InfoNCE)

The training objective is a symmetric cross-entropy contrastive loss over a $B \times B$ similarity logit matrix. Diagonal entries are positive pairs; all off-diagonal entries are negatives.

$$\mathcal{L} = \frac{1}{2}\left[\text{CE}\left(\frac{\mathbf{V}\mathbf{T}^\top}{\tau},\, \mathbf{I}\right) + \text{CE}\left(\frac{\mathbf{T}\mathbf{V}^\top}{\tau},\, \mathbf{I}\right)\right], \qquad \tau = 0.07 \tag{3}$$

where $\mathbf{V} \in \mathbb{R}^{B\times 512}$ and $\mathbf{T} \in \mathbb{R}^{B\times 512}$ are ℓ_2-normalized video and text feature matrices.

6.4 Training Configuration

Table 3: Stage 1 training hyperparameters.

Parameter	**Value**
Optimizer	Adam ($\text{lr} = 1 \times 10^{-5}$)
Epochs	3
Batch Size	2 (effective 4 with gradient accumulation)
Grad. Accumulation	Every 2 batches
Mixed Precision	`torch.cuda.amp.GradScaler` + `autocast`
Gradient Clipping	Max norm = 1.0
Temperature τ	0.07
Checkpoint	`stage1_clip_m.pht`

7. Stage 2 — Query-Dependent Detection Transformer (QD-DETR)

7.1 Architecture Overview

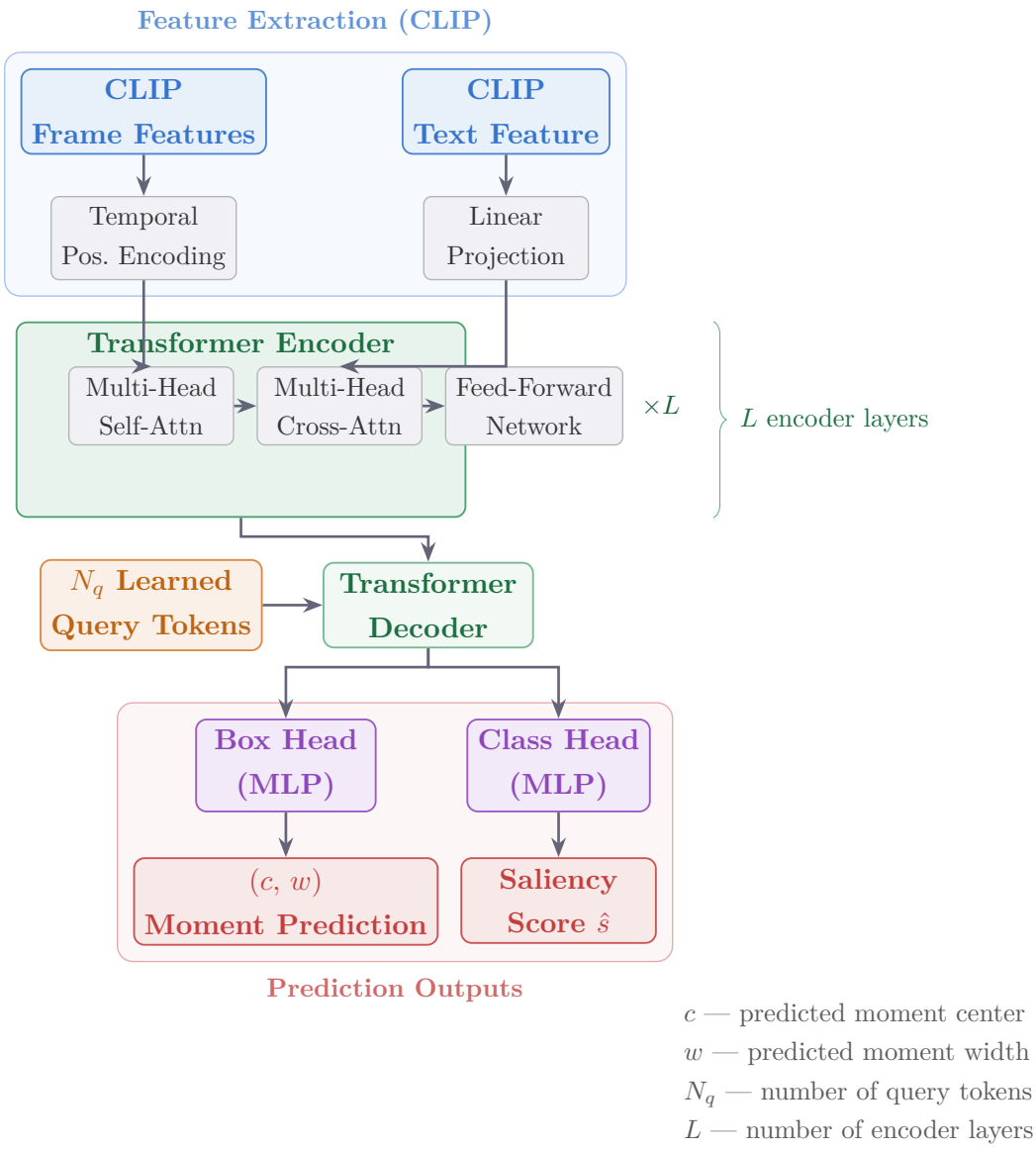
QD-DETR is a DETR-style model adapted for 1-D temporal grounding. It takes as input:

- $\mathbf{V} \in \mathbb{R}^{1\times N_f \times 512}$ — CLIP visual features for $N_f = 32$ uniformly sampled frames from the candidate segment.
- $\mathbf{q} \in \mathbb{R}^{1\times 512}$ — ℓ_2-normalized CLIP text feature.

A Transformer encoder encodes the visual token sequence; a set of $N_q = 20$ learnable query tokens attend to the encoded sequence via cross-attention. Each output query is decoded into:

- A 2-D box prediction $(c,\, w) \in [0, 1]^2$ (normalized center and width).
- A class logit vector for foreground/background classification.

tikz amsmath xcolor



Detailed QD-DETR block diagram. The text feature conditions the cross-attention of each query token, making predictions query-dependent.

7.2 Feature Extraction for Training

Before training QD-DETR, CLIP features are pre-extracted for every training sample and saved as `.pt` files to avoid recomputing them each epoch. Each file contains:

```
torch.save({
    "video": video_feat,    # [T, 512]   L2-normalized frame features
    "text" : text_feat,     # [512]      L2-normalized text feature
    "gt"   : gt,            # [2]        tensor([center, width]) in [0,1]
}, f"clip_features_1/{i}.pt")
```

Listing 2: Saved feature dictionary per sample.

Ground-truth boxes are converted to normalized format:

$$c = \frac{s/D + e/D}{2}, \qquad w = \frac{e - s}{D} \tag{4}$$

where s, e are ground-truth start/end times and D is the video duration.

7.3 Training Objective

QD-DETR is trained with a bipartite matching loss (Hungarian algorithm) that assigns each ground-truth box to the best-matching predicted query:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{cls}}\,\mathcal{L}_{\text{cls}} + \lambda_{\text{L1}}\,\mathcal{L}_{\text{L1}} + \lambda_{\text{IoU}}\,\mathcal{L}_{\text{IoU}} \tag{5}$$

7.4 Inference

During inference, the QD-DETR forward pass produces $N_q = 20$ candidate boxes. The box with the highest foreground logit score is selected:

$$k^* = \arg\max_k \; \hat{p}_k^{\text{fg}}, \qquad [\hat{s},\ \hat{e}] = \left[u_s + \left(c_{k^*} - \tfrac{w_{k^*}}{2}\right)\Delta,\ \ u_s + \left(c_{k^*} + \tfrac{w_{k^*}}{2}\right)\Delta\right] \tag{6}$$

where $\Delta = u_e - u_s$ is the candidate segment duration from Stage 1.

8. Evaluation Metrics

8.1 Temporal Intersection over Union (IoU)

For a predicted segment $[\hat{s}, \hat{e}]$ and ground-truth $[s, e]$:

$$\text{IoU} = \frac{\max(0,\ \min(\hat{e}, e) - \max(\hat{s}, s))}{\max(\hat{e}, e) - \min(\hat{s}, s)} \tag{7}$$

8.2 Recall@1 at IoU Threshold θ

$$\text{R@1, IoU} = \theta \ = \ \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\text{IoU}_i \geq \theta] \tag{8}$$

Reported at $\theta \in \{0.3,\ 0.5,\ 0.7\}$.

8.3 Mean IoU (mIoU)

$$\text{mIoU} = \frac{1}{N} \sum_{i=1}^{N} \text{IoU}_i \tag{9}$$

9. Experimental Results

9.1 CLIP Threshold Tuning (Val-1)

The CLIP similarity threshold was swept over the range $\{0.10, 0.12, 0.14, 0.16, 0.17, 0.18, 0.20, 0.22, 0.25\}$ on 400 val_1 samples. The optimal threshold was selected by maximizing R@1 at IoU= 0.5.

9.2 Final Results on test data Val-2

Table 4: Final evaluation results on ActivityNet Captions validation splits.

Method / Split	**R@1 IoU=0.3**	**R@1 IoU=0.5**	**R@1 IoU=0.7**	**mIoU**
Our model (val_2)	**0.77**	**0.483**	**0.0967**	**0.4371**

9.3 IoU Distribution

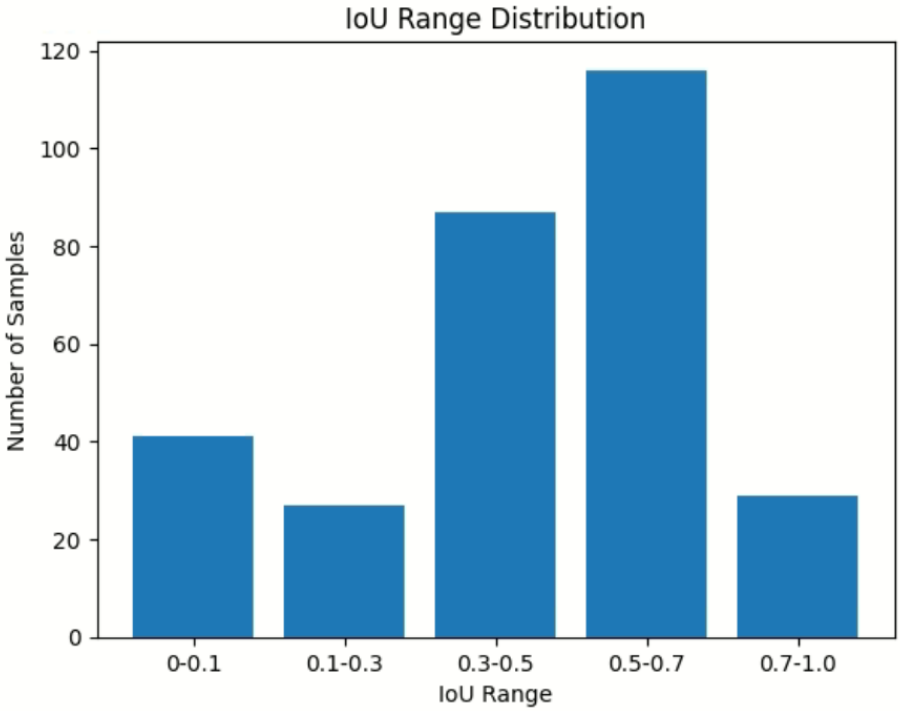


Figure 1: Your caption here

9.4 Recall vs. IoU Threshold Curve

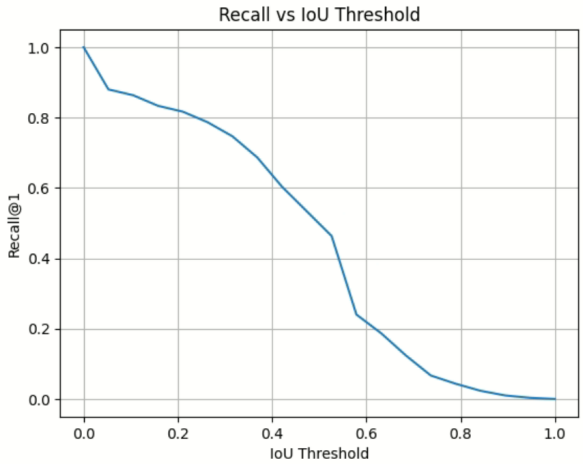


Figure 2: Recall@1 as a function of IoU threshold. The area under this curve summarises overall grounding quality.

9.5 Success vs. Failure Analysis

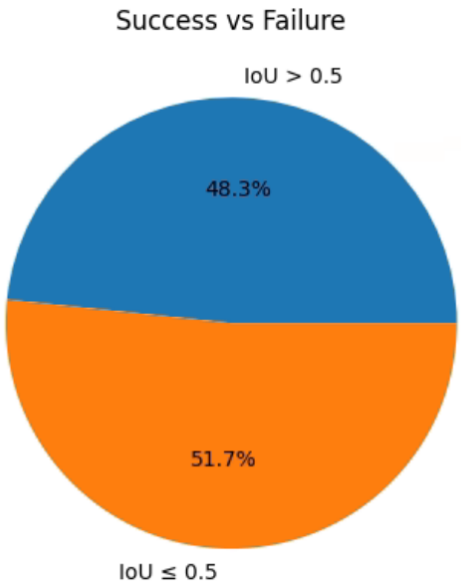


Figure 3: Success vs. failure at IoU threshold 0.5.

Figure 4: IoU range distribution (bar chart).

10. Implementation Details

10.1 Robustness & Error Handling

The feature extraction pipeline wraps all operations in `try/except` blocks with per-sample logging. NaN detection is applied to both video and text features after encoding; corrupted samples are skipped and logged to `failed_samples.txt`. Frame indices are always clipped to $[0, \text{len(vr)} - 1]$.

10.2 Resumability

Feature extraction can be resumed from any interruption point: files already present in the output directory are skipped based on filename lookup, avoiding redundant computation on large datasets.

10.3 Normalization

Both video and text feature vectors are ℓ_2-normalized (with a small $\epsilon = 10^{-6}$ for numerical stability) before any similarity computation or storage:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2 + \varepsilon} \tag{10}$$

11. Discussion

11.1 Strengths

- **Modular design**: Stage 1 and Stage 2 are independently trainable and evaluable.
- **Lightweight**: No optical flow, no 3D convolutions; only CLIP RGB features.
- **Efficient inference**: Pre-extracted features reduce Stage 2 forward-pass cost.
- **Threshold tunability**: The sliding-window threshold provides a precision-recall trade-off knob without retraining.

11.2 Limitations

- **Small batch size**: A batch size of 2 limits the diversity of negatives in contrastive learning; a larger GPU or gradient accumulation over more steps would improve alignment.
- **Fixed window size**: The 4-second sliding window may miss very short (< 2s) or very long (> 60s) events; adaptive window strategies could help.
- **Single-span prediction**: QD-DETR currently returns the single highest-scoring box; multi-span grounding (multiple events per query) is not supported.

11.3 Future Work

- Integration of temporal positional encodings into QD-DETR.
- Cross-modal fusion layers between video and text before the detection head.
- Evaluation on additional benchmarks (Charades-STA, TACoS).
- Larger CLIP backbone (ViT-L/14) for improved feature quality.

12. Conclusion

This report presented the Smart Video Indexer system, a two-stage pipeline that combines contrastive CLIP fine-tuning with a Query-Dependent Detection Transformer for precise temporal localization of natural language queries in untrimmed videos. The system was trained and evaluated on ActivityNet Captions, a challenging large-scale benchmark. The modular architecture allows independent improvement of each stage. Future work will focus on scaling the backbone, improving the detection head, and extending evaluation to additional grounding benchmarks.

A. Code Snippets

A.1 Attention Pooling

```
def attention_pooling(frame_features, text_features):
    # frame_features: [B, T, 512]
    # text_features:  [B, 512]
    sim     = torch.matmul(frame_features,
                           text_features.unsqueeze(-1)).squeeze(-1)  # [
                              B, T]
    weights = F.softmax(sim, dim=1)                                   #
       [B, T]
    video   = torch.sum(frame_features * weights.unsqueeze(-1), dim=1)
    return video                                                       #
        [B, 512]
```

Listing 3: Attention pooling over frame features.

A.2 Contrastive Loss

```
def contrastive_loss(similarity):
    labels  = torch.arange(similarity.size(0)).to(device)
    loss_v2t = F.cross_entropy(similarity,   labels)
    loss_t2v = F.cross_entropy(similarity.T, labels)
    return (loss_v2t + loss_t2v) / 2
```

Listing 4: Symmetric InfoNCE contrastive loss.

A.3 Metrics Computation

```python
def iou_1d(pred_s, pred_e, gt_s, gt_e):
    inter = max(0, min(pred_e, gt_e) - max(pred_s, gt_s))
    union = max(pred_e, gt_e) - min(pred_s, gt_s)
    return inter / union if union > 0 else 0.0

def compute_metrics(results):
    ious = [r["iou"] for r in results]
    return {
        "R@1_IoU0.3" : sum(i >= 0.3 for i in ious) / len(ious),
        "R@1_IoU0.5" : sum(i >= 0.5 for i in ious) / len(ious),
        "R@1_IoU0.7" : sum(i >= 0.7 for i in ious) / len(ious),
        "mIoU"       : sum(ious) / len(ious),
        "recall"     : sum(r["found"] for r in results) / len(results),
        "n_evaluated": len(results),
    }
```

Listing 5: Temporal IoU and evaluation metrics.